2 Requirements engineering

Week 2 · Week summary

Core ideas, lesson by lesson

1 What a requirement is, and why it comes first

- A requirement says what the system must do or how well, not how it is built.
- Clients bring problems; you find the causes and derive requirements from them.
- Requirements = wishes adjusted to physical, legal, financial, social, and security constraints.
- The later a requirements error is found, the more it costs to fix.

2 Types of requirements: functional, non-functional, user, and system

- Functional = what the system does. Non-functional = how well it does it, or under which constraints.
- Non-functional families: product (efficiency, usability, reliability, portability, security), organisational, external.
- FURPS+: Functionality, Usability, Reliability, Performance, Supportability, plus constraints.
- Every non-functional requirement needs a number you can test.
- User requirements speak to clients; system requirements give developers the detail.

3 The requirements engineering process and its people

- Core activities: feasibility, elicitation, analysis, specification, validation, management.
- The activities overlap and repeat; management runs the whole time.
- A stakeholder is anyone affected by the system or able to influence it, even without using it.
- The analyst asks what, where, whose, why, how, when, and what could stop us.

4 Elicitation and analysis: gathering and refining needs

- Techniques: interviews, questionnaires, observation, workshops, document analysis, prototypes, scenarios.
- Three difficulties: scope, understanding, and volatility.
- Analysis refines, splits, merges, finds conflicts, and checks feasibility.
- Business policies are the client's decisions: make them explicit and keep them configurable.

5 Specification and validation: writing requirements people can check

- An SRS has an introduction, an overall description, specific requirements, and appendices.
- One idea per numbered requirement, consistent words, a glossary, a source, and a priority.
- Quality: correct, unambiguous, complete, consistent, verifiable, feasible, traceable, modifiable.
- Validate with reviews, prototypes, and early test cases, always with the client.

6 Managing change and tracing requirements

- Enduring requirements are stable; volatile ones change (changing, emergent, consequential, compatibility).
- Every change: record, analyse impact, approve or reject, apply, new version.
- Traceability: source (where it came from), requirements (what depends on it), design (what implements and tests it).

7 User stories: requirements as conversations

- Template: As a <role>, I want <goal>, so that <benefit> (who, what, why).
- Three Cs: Card, Conversation, Confirmation.
- INVEST: Independent, Negotiable, Valuable, Estimable, Small, Testable.
- Theme > epic > story > task. Tasks say how; stories say what and why.
- The backlog is ordered by value; MoSCoW sorts a release into Must, Should, Could, Won't.

8 Acceptance criteria, Given–When–Then, and splitting stories

- Acceptance criteria say when a story is done; agree on them before work starts.
- Given (context), When (event), Then (expected result); And adds more.
- Cover the happy path and the important unhappy paths with real example values.
- More than about five criteria? Split the story: by steps, rules, paths, roles, data, operations.



Watch out

Mistake	Fix
Writing how to build it (database, framework) instead of what the user needs.	Ask "what can the user do or rely on?" and move technology choices to design.
Non-functional requirements without numbers ("fast", "secure").	Add a measure and a target that a test can check.
User stories without the "so that" or without acceptance criteria.	Always write the benefit, and agree Given–When–Then scenarios, edge cases included, before work starts.
Treating the first thing the client says as the requirement.	Ask "why?" until you reach a cause you can act on, then write the requirement for that cause.
Mixing solutions into requirements ("use Firebase", "add a dropdown").	Describe the need ("the reader picks the branch where they will collect the book") and leave the how to design.
Writing non-functional requirements with adjectives: fast, secure, user-friendly.	Pick a measure and a target: "a new user books a class in under 60 seconds without help".
Calling security or privacy "functional" because there is a login screen.	The login screen is functional; the rule "passwords are stored hashed and sessions expire after 15 minutes idle" is a security quality.
Interviewing only the person who pays.	List every stakeholder group first; desk staff know problems the council has never seen.